

Git Guild P4 演示说明

最后更新：2026-06-07

1. 演示目标

本次演示使用本地可运行版本展示 Git Guild 的 P4 核心闭环：

维护者发布悬赏任务 -> 管理员审核上架 -> 冒险家接取任务
-> 工作台克隆并推送代码 -> 提交柜台自动创建 PR
-> 维护者审核通过 -> PR 自动合并 -> XP 与成长记录更新

2. 本地环境

项目	地址或版本	说明
前端	http://localhost:5173	Vue 3 + Vite，本地开发服务器
后端	http://localhost:8080	Spring Boot 3，REST API
Swagger UI	http://localhost:8080/swagger-ui.html	API 调试入口
OpenAPI JSON	http://localhost:8080/api-docs	API 契约 JSON
MySQL	localhost:3307	Docker Compose 暴露端口，容器内为 3306
Redis	localhost:6379	站内通知和缓存支撑
Gitea	http://localhost:3000	本地代码托管底座

3. 还原干净演示版本

在仓库根目录执行：

```
cd D:\aaaWW\SoftwareEngineII\project\Git-Guild
docker compose down -v
docker compose up -d
docker compose ps
```

down -v 会删除 MySQL、Redis、Gitea 数据卷；下一次 up -d 会重新从仓库内置快照恢复数据：

快照文件	自动恢复位置	说明
seed/mysql-seed.sql	MySQL gitguild 数据库	预置 Git Guild 账号、仓库、Issue 等业务数据
seed/gitea-data.tar.gz	Gitea 数据卷	预置 Gitea 管理员账号和演示仓库

确认 `gitguild-mysql`、`gitguild-redis`、`gitguild-gitea` 均为 `healthy` 后，再启动后端和前端。

4. 启动方式

4.1 启动后端

```
cd D:\aaaWW\SoftwareEngineII\project\Git-Guild\backend
.\mvnw.cmd spring-boot:run
```

演示快照对应的 Gitea token 已作为本地默认配置写入后端配置，正常演示不需要额外生成 `.env`。该 token 只用于本机 `localhost:3000` 的一次性演示环境，不应作为生产认证方案。

4.2 启动前端

```
cd D:\aaaWW\SoftwareEngineII\project\Git-Guild\frontend
npm install
npm run dev
```

Vite 已配置代理：

- `/api -> http://localhost:8080`
- `/uploads -> http://localhost:8080`

浏览器访问 `http://localhost:5173` 即可进入演示界面。

5. 预置账号与数据

5.1 Git Guild 账号

当前干净快照中已经预置 3 个 Git Guild 账号，密码均为 `admin123`：

演示角色	用户名	邮箱	密码	后端角色
管理员	admin	admin@gg.local	admin123	ADMIN

演示角色	用户名	邮箱	密码	后端角色
委托人 / Guild Master	guild	guild@gg.local	admin123	MAINTAINER
冒险家 / Adventurer	advent	advent@gg.local	admin123	BEGINNER

说明：前端入口中“冒险家”对应后端初始角色 BEGINNER。

5.2 Gitea 账号与仓库

项目	当前快照值
Gitea 地址	http://localhost:3000
Gitea 管理员用户名	spike-admin
Gitea 管理员邮箱	spike@gitguild.local
是否管理员	是
演示仓库	hello-world-mvp
仓库 URL	http://localhost:3000/spike-admin/hello-world-mvp
预置 Issue	#1 Add hello world file

5.3 Git Guild 业务数据

数据类型	当前快照值
仓库	repository_id=1，hello-world-mvp，状态 SYNCED
Issue	issue_id=1，Gitea external_id=1，标题 Add hello world file，状态 OPEN
任务分类	MVP
悬赏任务	初始为空，需要在演示中由 guild 发布
成果提交 / PR / XP 记录	初始为空，随演示流程生成

6. 核心演示路径

6.1 维护者发布任务

1. 打开 http://localhost:5173。
2. 使用 guild 登录。
3. 进入发布悬赏任务入口。

4. 使用预置仓库和 Issue：

```
仓库: hello-world-mvp  
仓库地址: http://localhost:3000/spike-admin/hello-world-mvp  
Issue: #1 Add hello world file
```

5. 填写任务信息并提交。

6.2 管理员审核上架

1. 使用 `admin` 登录。
2. 打开管理员审核入口。
3. 找到刚发布的任务。
4. 选择审核通过。

6.3 冒险家接取任务

1. 使用 `advent` 登录。
2. 进入 Quest Board / 悬赏任务板。
3. 打开刚发布的任务详情。
4. 点击接取任务。

6.4 工作台克隆并推送代码

1. 进入冒险家工作台。
2. 使用 Clone & Push 卡片中的克隆地址。
3. 按页面提示克隆仓库、切换任务分支、修改文件、提交并推送。

说明：演示用克隆地址会携带本地 Gitea 凭据或 token，用于避免 `git push` 时再输入密码。该方式只适用于本机演示环境。

6.5 提交柜台自动创建 PR

1. 冒险家进入提交柜台。
2. 选择当前任务。
3. 填写成果说明和自检清单。
4. 提交成果。

6.6 维护者审核成果

1. 使用 `guild` 登录。
2. 进入维护者审核队列。

- 3. 打开冒险家的成果提交。
- 4. 点击审核通过。

6.7 成长档案与排行榜

- 1. 使用 `advent` 登录。
- 2. 打开个人档案或成长信息页面。
- 3. 查看 XP、等级、完成任务数和贡献记录。
- 4. 打开排行榜查看排名变化。

7. 异常分支演示

异常场景	操作	预期结果
未登录访问受保护页面	直接访问工作台、提交柜台或管理员页面	跳转登录页或提示需要登录

8. 本次干净快照检查记录

检查时间：2026-06-07 20:47，本地执行 `docker compose down -v` 后重新 `docker compose up -d`。

检查项	结果
Docker 容器	MySQL、Redis、Gitea 均为 <code>healthy</code>
Git Guild 用户	仅预置 <code>admin</code> 、 <code>guild</code> 、 <code>advent</code> 三个演示账号
Git Guild 仓库	<code>hello-world-mvp</code> ，状态 <code>SYNCED</code>
Git Guild Issue	<code>#1 Add hello world file</code> ，状态 <code>OPEN</code>
Git Guild 任务	<code>quests=0</code> ，等待演示时发布
Gitea 用户	<code>spike-admin</code> ，邮箱 <code>spike@gitguild.local</code> ，管理员

后续如需重新开始演示，重复执行：

```
docker compose down -v
docker compose up -d
```